

Agentic AI for PMs and TPMs

Week 7

Fine-Tuning of Agents

Fine-Tune a 2-Agent Pipeline to Eliminate Hallucination

Instructor :

A little about me, Joy Chakraborty

- Sr. Technical Program Manager, Agentic AI Foundation
- Machine Learning and Robotics for last 15+ years
- Owns leading top 2 Agentic AI Products
- Speak at Public Conferences
- Contribute to the Open-Source Projects



joychak1



joychak1@yahoo.com

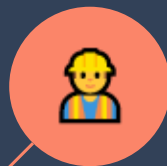
Welcome to today's class, before we begin, pop into the chat:



Hannah
Chen



Data
Scientist



Bay Area



Grammarly, ex-
Uber



Today's Agenda

★

1



Fine Tuning Pipeline

Break

2



Demo in N8N/HF/OpenAI

Break

3



Decoding the Demo

What Is Fine-Tuning?

Prompt engineering tells an agent what to do. Fine-tuning teaches it how correct behavior looks.

Fine-Tuning Training a base LLM on curated examples so it learns behaviors, tone, and domain rules — baked in, not prompted in. Model weights change. Behavior is internalized.

01 Prompting

Runtime Instructions

Rules in the system message. Fast to change. But the model re-reads them every call — can deviate on edge cases and vague inputs.

02 Fine-Tuning

Internalized Behavior

Training reshapes model weights. The model doesn't need to re-read rules — it has seen examples. Works reliably on unexpected inputs.

03 RAG

External Knowledge

Retrieves documents at query time. Adds facts the model doesn't know. Doesn't change reasoning or output format — only knowledge.

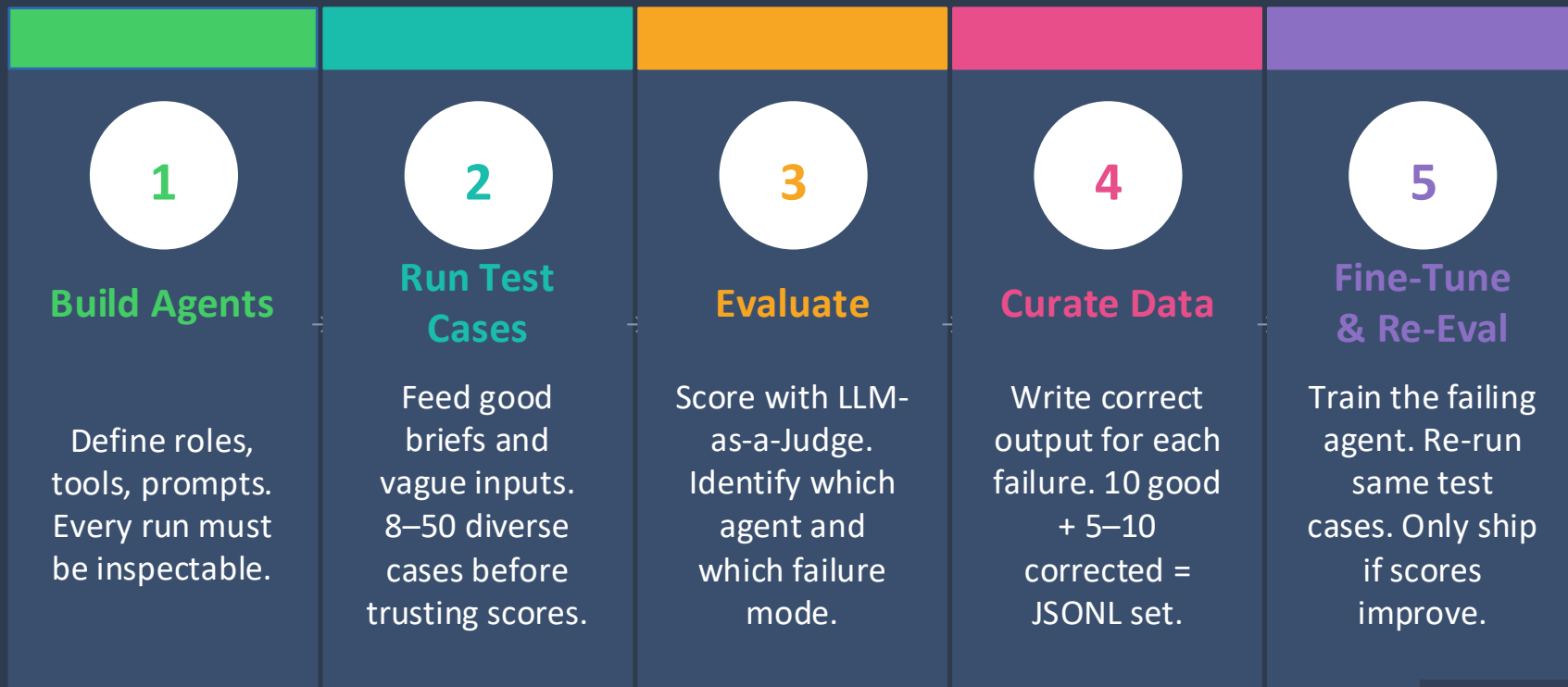
★ Fine-tuning is the difference between an agent that reads instructions and one that has internalized them.



INTERVIEW
KICKSTART

The Full Journey: Build → Evaluate → Fine-Tune

Fine-tuning is never step one. It is the last optimization lever — after evaluation surfaces a clear, repeating failure.



★ You cannot fine-tune your way out of a badly designed pipeline. Build and evaluate first — always.

The Fine-Tuning Spectrum: When to Use Each Lever

You have four levers. Fine-tuning is the most powerful — and most expensive. Exhaust the others first.

START HERE	KNOWLEDGE	★ TODAY	TEST FIRST
01	02	03	04
Prompt Engineering	RAG / Knowledge	Fine-Tuning	Model Swapping
Impact: High Cost: Low Risk: Low Change instructions, add few-shot examples, output constraints. Zero training cost. Test in minutes.	Impact: High Cost: Low Risk: Low Agent needs external facts. Doesn't change behavior — only knowledge. Use before fine-tuning domain facts.	Impact: Highest Cost: Medium Risk: Med Prompt fixes have plateaued. Behavior must be consistent on edge cases. Requires labeled data. Validate before ship.	Impact: Variable Cost: Lo–Med Risk: Med Replace the LLM for a specific agent. Always validate with your evaluation dataset before shipping.

★ Order: Prompt → RAG → Fine-Tune → Swap. Each step has higher cost and commitment than the last.



INTERVIEW
KICKSTART

Fine-Tuning Decisions for PMs/TPMs

Fine-tuning is a product decision — with cost, quality, and risk implications.



Cost & ROI Decision

Upfront data prep cost but inference savings at scale. Is the quality delta worth it? Wrong timing = expensive with no payback.



Quality Bar Definition

Only a PM knows when 'good enough' is actually good enough. Fine-tuning targets must tie to product quality bars — not model benchmarks.



Invisible Failure Risk

A fine-tuned model overfit on narrow training data fails on new inputs. Without PM-defined test cases, this ships to users undetected.



Data Strategy is Yours

Fine-tuning is only as good as the training data. PMs define what correct looks like. Engineers execute the training. Source of truth lives with PM.

★ If you can't define what correct looks like, you can't fine-tune. Data curation is a PM responsibility.



INTERVIEW
KICKSTART

The Problem Fine-Tuning Solves: Hallucination

The base model fills vague inputs with confident fiction. Fine-tuning teaches it to flag UNCLEAR instead.

✓ Detailed Input — Behaves Well

"Checkout freezes applying discount on iPhone 14, Safari. Happens every time."

- ✓ category: Bug Report
- ✓ affected: Checkout UI
- ✓ platform: iOS / Safari
- ✓ severity: HIGH — reproducible

Works fine — no fine-tuning needed.

AI
FILLS
GAPS

⚠ Vague Input — Hallucination

"This app is garbage. Total waste of money."

Base model INVENTS:

- ✗ affected: 'API Gateway' (fabricated)
- ✗ root_cause: 'Memory leak' (invented)
- ✗ priority: P1 Critical (hallucinated)

Fine-tuned model produces instead:

*"category: UNCLEAR
affected: UNKNOWN — Insufficient detail
for engineering action."*

What Is Training Data? The JSONL Format

Each example is a complete conversation. The model learns: given this input, produce exactly this output.

training_data_classifier.jsonl — one line = one example

```
{"messages": [
  {"role": "system",      "content": "You are a Feedback Classifier..."},
  {"role": "user",        "content": "This app is garbage. Total waste."},
  {"role": "assistant",   "content": "{ \"category\": \"UNCLEAR\",
                                   \"affected_system\": \"UNKNOWN\" }"}
]}
```

System Prompt

Must match your production system message exactly.
Consistency across all examples is critical.

User Message

The input — detailed or vague.
Include both types. Vague inputs are the key training signal.

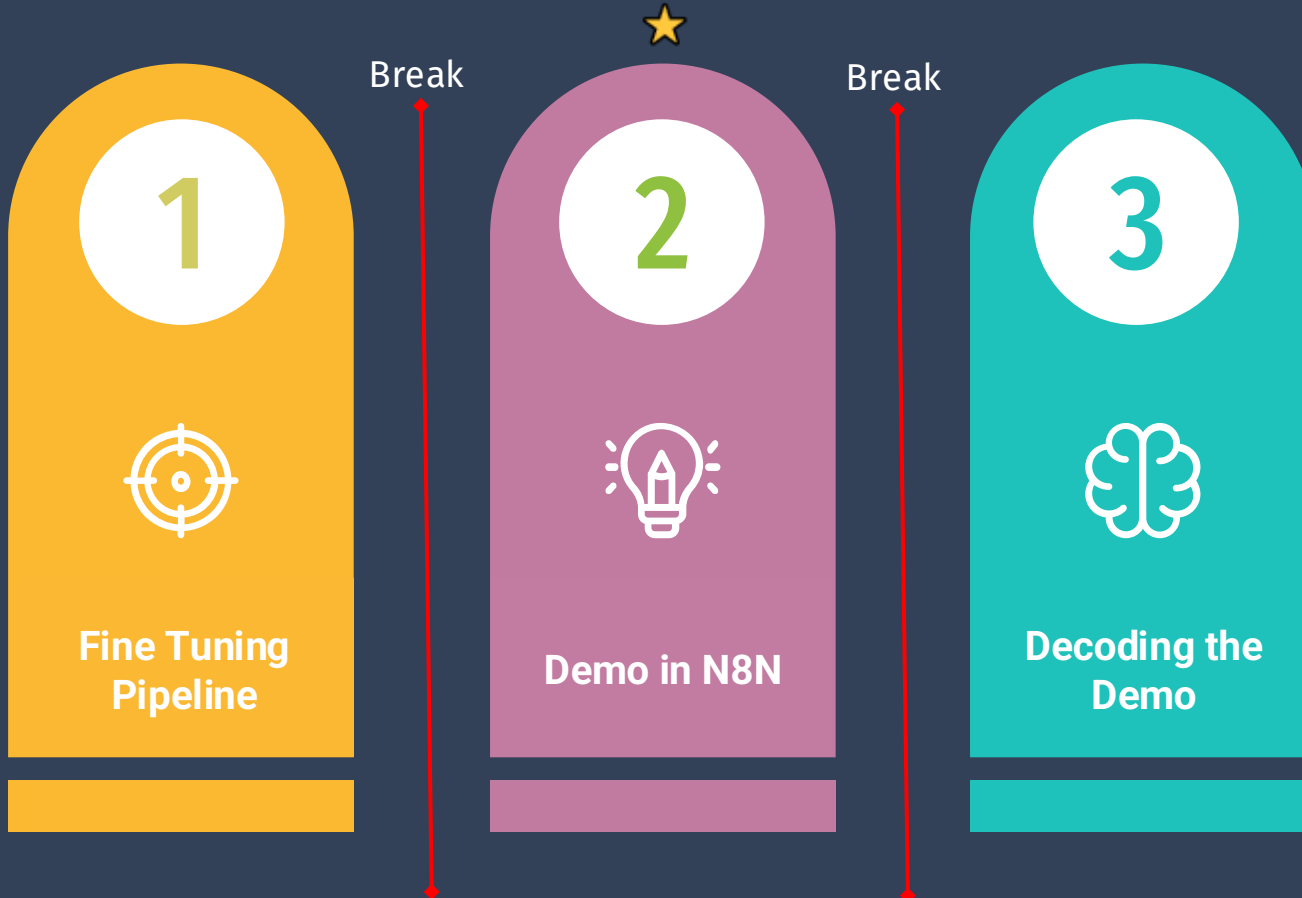
Assistant Output

The correct answer. Vague input → UNCLEAR / UNKNOWN. Never invented values.

Key ratio: 10 good examples + 5–10 corrected examples = 15–20 total. Today's demo uses exactly 15 per agent.

★ Every vague input that gets a corrected output is a training example. Start labeling from day 1.

Today's Agenda



Demo Overview: What We're Building & Why

Before we run the demo — here is the problem, the tools, and what success looks like.

The Problem

A 2-agent pipeline processes raw user feedback. The base model invents engineering details whenever feedback is vague — fabricated API names, made-up components. Expensive, trust-destroying, invisible without evaluation.

Pipeline Orchestrator

n8n

Visual workflow builder. Agent 1 = Classifier, Agent 2 = Insight Writer. Sends outputs to DeepEval for scoring. Fully traceable.

Fine-Tuning Engine

Llama-3.1-8B-Instruct

Hosts base model and fine-tuned versions. Upload JSONL → get back a custom model ID to swap in.

Evaluation Framework

DeepEval

GPT-4 as Judge. Scores both runs — base and fine-tuned — across 4 metrics. Numeric evidence of improvement.

What Success Looks Like

Hallucination score drops. Vague Feedback Handling flips FAIL → PASS. Measured on the same 8 test cases before and after fine-tuning.

★ Same inputs. Same metrics. Different model. Improvement must be measurable — not just 'feels better'.



LIVE DEMO

Fine-Tuning a 2-Agent Feedback Pipeline in n8n + OpenAI

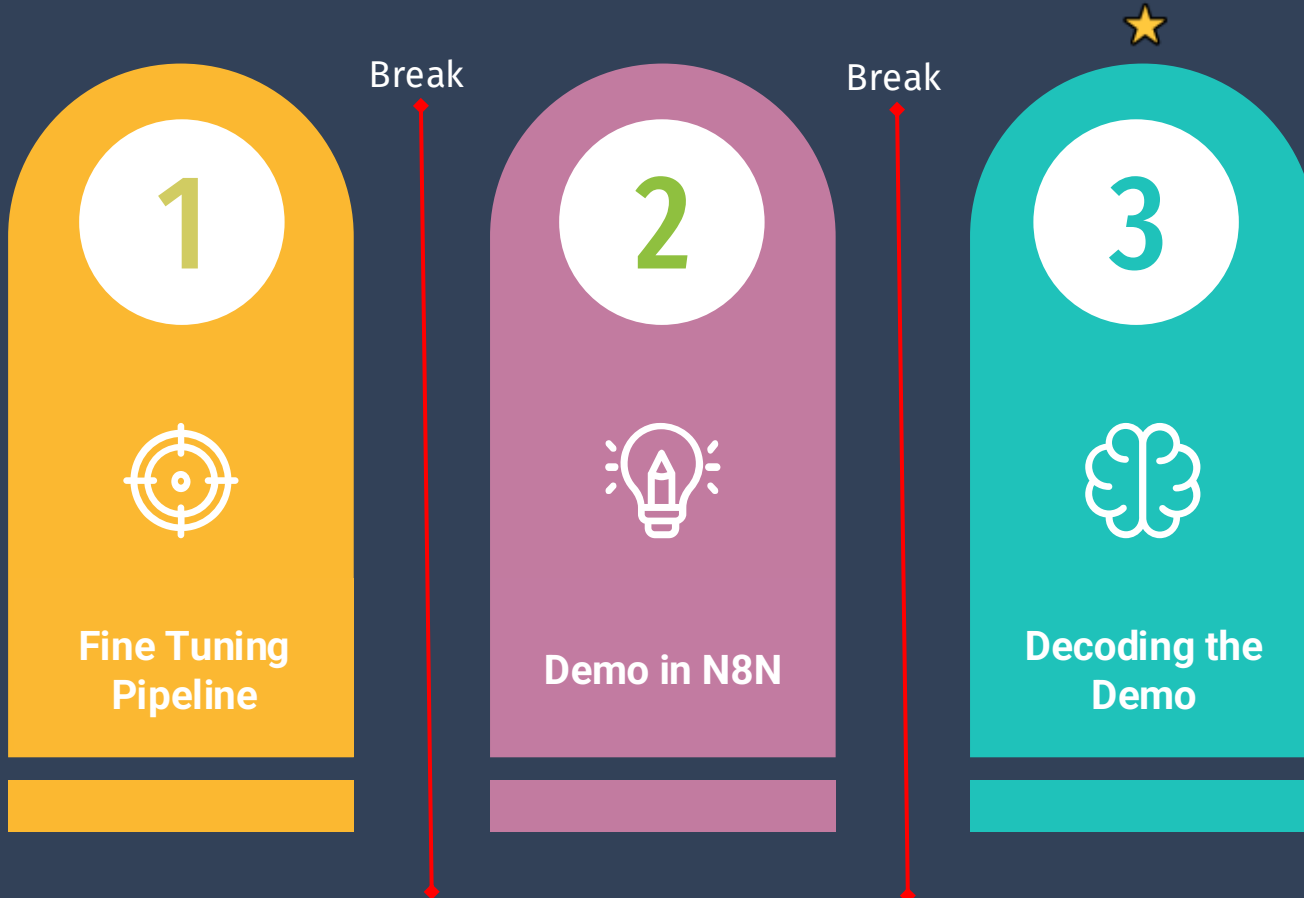
Evaluated with DeepEval · Fine-tuned with OpenAI API · Improvement measured

Watch for: The base model invents engineering details from vague feedback — fine-tuning fixes this completely.



INTERVIEW
KICKSTART

Today's Agenda



Demo Setup: The 2-Agent Pipeline Architecture

Two specialized agents in series. Each fine-tuned independently with its own training data, evaluation criteria, and failure modes.



DeepEval — Evaluation Layer (GPT-4 as Judge)

4 metrics × 8 test cases: Engineering Groundedness · Engineering Usefulness · Vague Feedback Handling · Answer Relevancy

Before vs After: What the Scores Show

Same 8 test cases. Same 4 metrics. Different model. The improvement is measurable — not subjective.

Evaluator / Metric	Before Fine-Tuning	After Fine-Tuning	Result
Engineering Groundedness	Low — invents API names, system components	High — only references input text	✅ FIXED
Vague Feedback Handling	FAIL — fabricates tickets from vague input	PASS — flags UNCLEAR, no invented content	✅ FIXED
Engineering Usefulness	Mixed — bad on vague inputs	Consistently high across all 8 test cases	✅ IMPROVED
Answer Relevancy	Medium — disconnected from vague inputs	High — directly addresses what was said	✅ IMPROVED

★ Evaluate → Identify failure → Curate data → Fine-tune → Re-evaluate. This loop is the engine of improvement.

Why DeepEval? — The Evaluation Tool Explained

DeepEval gives automated, metric-driven scoring so we can measure improvement — not just feel it.

What is DeepEval?

Open-source Python framework. Uses GPT-4 as LLM-as-a-Judge to score pipeline outputs across quality dimensions — no manual review needed at scale.

Why not just eyeball outputs?

Eyeballing is subjective and doesn't scale. DeepEval gives repeatable numeric scores before AND after fine-tuning — so improvement is provable to stakeholders.

The 4 Metrics Used in Today's Demo

Engineering Groundedness

Does output only reference input facts? Catches fabricated API names and system components.

Vague Feedback Handling

Does agent flag UNCLEAR on vague input instead of fabricating? The core hallucination test.

Engineering Usefulness

Is the output actionable for a dev team? Evaluates ticket quality and repro step accuracy.

Answer Relevancy

Is the response tied to what was said? Penalises outputs disconnected from the input.

★ DeepEval turns 'it looks better' into 'hallucination dropped 40%' — that is a shippable result.



INTERVIEW
KICKSTART

Why did we fine-tune each agent separately?

Think about it: Could one combined model do both classification and ticket writing at once?

Decoding the Demo

Why Fine-Tune Each Agent Separately?

Could we fine-tune one combined model to do both jobs at once?

Different Jobs, Different Output Formats

Classifier → structured JSON. Insight Writer → narrative ticket. One model trained on both averages them — and does both badly.

Independent Improvement Loops

If the Classifier degrades, fix only its training data. A combined model forces retraining everything when one part breaks.

Modular Evaluation = Precise Debugging

DeepEval scores Agent 1 independently of Agent 2. You know exactly which agent caused which failure.

Cost-Efficient Training

Two focused 15-example datasets train faster and converge better than one 30-example mixed dataset.

★ Rule: one fine-tuning job per agent role. Same discipline as one system prompt per agent.

Why didn't we just add more prompt guardrails?

Think about it: The system prompt said 'Do NOT invent facts.' Why didn't that work?

Why Prompt Guardrails Weren't Enough

The system prompt said 'Do NOT invent facts.' Why didn't it work on vague inputs?

Instructions Are Read, Not Internalized

A prompt says 'don't hallucinate.' A fine-tuned model has seen 5 examples of what to do instead. Seeing correct behavior beats reading one rule.

Long Prompts Lose Attention

The more guardrails you add, the less reliably each one is followed. Models lose focus in long instruction lists on edge cases.

Prompts Don't Generalize to Novel Inputs

A guardrail for 'app is garbage' may fail for 'meh, fine I guess.' Fine-tuning teaches the general principle. Prompts teach one case at a time.

Prompts + Fine-Tuning = Production Stack

In production you use both. Prompts define context and role. Fine-tuning bakes in behavioral patterns. Today's agents have both.

What if I only have 5 training examples? Is fine-tuning worth it?

Think about it: Data volume is one factor — but quality and diversity matter more.

Decoding the Demo

What If I Only Have 5 Training Examples?

Data volume is one factor — quality and diversity matter more.

5 Examples CAN Work — If They're the Right 5

5 corrected examples can shift behavior on a specific failure. Each must be 100% correct and cover a real edge case the base model always fails on.

When 5 Will NOT Be Enough

If your failure mode has many sub-variants, 5 examples won't cover the distribution. Narrow data = narrow fixes.

What Makes a Good Training Example?

- System prompt identical to production
- Input that is a real failure case
- Output completely correct
- Edge case the base model always fails on

The Minimum Viable Dataset

10 good examples (correct behavior on detailed input)
+ 5–10 corrected (fix the failure mode)
= 15–20 total. Today's demo uses exactly 15 per agent.

★ Start collecting labeled examples from day 1 — even before you fine-tune. Today's annotations are tomorrow's training data.

What if fine-tuning makes the model worse on some metrics?

Think about it: Regression is the most common fine-tuning failure. This is why you evaluate all metrics.

What If Fine-Tuning Causes Regression?

Regression is the most common fine-tuning failure — evaluate every metric, not just the one you fixed.

Q: Why does regression happen?

A: Model overfit to training examples and lost general capability. It memorized examples — didn't learn the principle. Happens when data is too narrow.

Q: How do you detect it early?

A: Run all metrics before and after — not just the one you were fixing. Regression shows as improvement on one + drop on another. You need the full picture.

Q: What do you do when it happens?

A: Don't ship. Add diverse examples covering regressed cases. Include correct behavior on degraded cases. Retrain. Re-evaluate. It's the loop.

Q: How do you prevent it?

A: Include both corrected examples AND good examples in every training run. Don't train only on failure cases. 10 good + 5–10 corrected is the proven ratio.

Q: What if scores are mixed?

A: Set a threshold BEFORE training: e.g. 'Hallucination <30% and no other metric drops >5%.' If threshold isn't met — it does not ship. No grey zones.

★ Set pass/fail thresholds BEFORE training. If any metric drops beyond threshold — it does not ship.

How do you know when you're done fine-tuning?

Think about it: A single iteration has clear exit criteria — define them before training begins.

How Do You Know When You're Done?

Exit criteria must be defined before you begin training — not after seeing the results.

Ready to ship when:	NOT done when:	The threshold rule:
Target metric improved by $\geq$ pre-set threshold	Only training examples score well	Set thresholds BEFORE training — not after
No other metric degraded beyond acceptable range	New edge cases still trigger old failure	e.g. Hallucination -30%, no metric down >5%
Model passes full evaluation dataset	Haven't run evaluation — judging by feel	Thresholds are a PM decision, not engineering
Failure mode gone from manual spot-checks	One improved but another regressed past threshold	Rationalizing mediocrity $\neq$ quality gate

★ Exit criteria before training = objective quality gate. Exit criteria after results = rationalization.

What if we skip evaluation and go straight to fine-tuning?

Think about it: This is the most expensive mistake in AI product development.

What If We Skip Evaluation?

Build → Evaluate → Fine-Tune is non-negotiable. Here's what happens when you skip step 2.

You don't know what you're fixing

Without baseline scores you're training blind. You might fine-tune for a failure that doesn't exist — or miss the actual one entirely.

You can't measure improvement

'Feels improved' is not a product quality bar. Stakeholders and users deserve measurable evidence. No baseline = no proof.

You train on the wrong failure mode

A model scoring 60 on Groundedness vs 40 on Usefulness needs different training data for each. Evaluating first tells you which to fix.

You create regression you can't detect

Without a pre-training baseline, you can't tell if fine-tuning made something worse. Invisible without the before-score.

★ **Evaluate first — always. Every fine-tuning job without a baseline is an opinion, not an improvement.**

The Fine-Tuning Flywheel: Evaluation Data Compounds

Today's evaluation data becomes tomorrow's training data. The flywheel compounds with every cycle.

① Evaluate

Traces + scores captured

Teams that instrument evaluation from week 1 have hundreds of labeled examples by the time they need to fine-tune. Don't wait for a crisis.

② Optimize

Fix prompts, measure gains

A score of 0.2 labels that output 'wrong.' A score of 0.9 labels it 'correct.' Scores create labels automatically — no extra work.

③ Fine-Tune

Train on scored traces

High-score traces reinforce good behavior. Low-score traces — with corrected outputs — fix bad behavior. Both sides of distribution matter.

④ Evaluate Again

Validate improvement

Re-run the same test cases. Compare score distributions. Measurable improvement = ship. Otherwise — back to step 2. The loop tightens.

★ The flywheel compounds — teams collecting labeled examples today will have an insurmountable data advantage.

Best Practices: Fine-Tuning Agents

These principles apply whether you are using OpenAI, Anthropic, Google, or any other fine-tuning platform.



DO

- ✓ Exhaust prompt engineering first
- ✓ Include corrected examples for each failure mode
- ✓ Evaluate before AND after with the same test set
- ✓ Fine-tune one model per agent role
- ✓ Set pass/fail thresholds before training
- ✓ Version your training data like code

X DON'T

- X Fine-tune without a baseline evaluation score
- X Train on hallucinated outputs — garbage in, garbage out
- X Mix agent roles in one training dataset
- X Deploy without re-running your full test suite
- X Skip negative examples — they are the most important
- X Change prompts AND fine-tune at the same time

Key Takeaways



Prompting tells agents what to do. Fine-tuning teaches them how correct behavior looks. You need both in production.



Build → Evaluate → Fine-Tune. This sequence is non-negotiable. You cannot fine-tune your way out of a badly designed pipeline.



Data curation is a PM responsibility. Only you can define what correct looks like for your domain. Engineers execute — PMs define quality.



Always evaluate before AND after with the same test set. Without a baseline you cannot prove improvement — or catch regressions.



Fine-tune one model per agent role. Separate jobs = separate evaluation, separate debugging, separate improvement loops.



Today's evaluation data is tomorrow's training data. Start collecting labeled examples now — the flywheel compounds over time.

Thank
you



joychak1@yahoo.com